

# PTI Fundamentos do Python 1

1

Qual é a saída do seguinte snippet?

```
1 my_list = [1, 2]
2
3 for v in range(2):
4     my_list.insert(-1, my_list[v])
5
6 print(my_list)
7
```

[1, 2, 2, 2]

[2, 1, 1, 2]

[1, 2, 1, 2]



[1, 1, 1, 2]

## Pergunta 2

O significado de um *argumento posicional* é determinado por:



sua posição na lista de argumentos

seu valor

sua conexão com variáveis existentes

o nome do argumento especificado junto com seu valor

## Pergunta 3

Quais das seguintes frases são **verdadeiras** sobre o código? (Selecione **duas** respostas)

```
1 | nums = [1, 2, 3]
2 | vals = nums
3 |
```



`nums` e `vals` são nomes diferentes da mesma lista



`nums` tem o mesmo comprimento que `vals`

`vals` é maior que `nums`

`nums` e `vals` são listas diferentes

## Pergunta 4

Um operador capaz de verificar se dois valores **não são iguais** é codificado como:

<>



!=

not ==

==

## Pergunta 5

O seguinte trecho:

```
1 def function_1(a):  
2     return None  
3  
4  
5 def function_2(a):  
6     return function_1(a) * function_1(a)  
7  
8  
9 print(function_2(2))  
10
```

produzirá 16



causará um erro de execução

## Pergunta 6

O resultado da seguinte divisão:

```
1 | 1 // 2
```

é igual a 0.0

não pode ser previsto



é igual a 0

é igual a 0.5

## Pergunta 7

O seguinte trecho:

```
1 def func(a, b):  
2     return b ** a  
3  
4  
5 print(func(b=2, 2))  
6
```

produzirá 2

produzirá 4



está errado

produzirá None

## Pergunta 8

Qual valor será atribuído à variável `x`?

```
1 | z = 0
2 | y = 10
3 | x = y < z and z > y or y < z and z < y
4 |
```

☐ 1

☐ True

☐ 0



False

## Pergunta 9

Quais dos seguintes nomes de variáveis são **ilegais** e causarão a exceção `SyntaxError`? (Selecione **duas** respostas)

☐ In



for

☐ print



in

## Pergunta 10

Qual é a saída do seguinte snippet?

```
1 my_list = [x * x for x in range(5)]
2
3
4 def fun(lst):
5     del lst[lst[2]]
6     return lst
7
8
9 print(fun(my_list))
10
```



[0, 1, 4, 9]

[0, 1, 9, 16]

## Pergunta 11

Qual é o resultado do seguinte trecho de código?

```
1 x = 1
2 y = 2
3 x, y, z = x, x, y
4 z, y, z = x, y, z
5
6 print(x, y, z)
7
```

1 2 1



1 1 2

1 2 2

2 1 2

## Pergunta 12

Qual será o resultado do seguinte snippet?

```
1 a = 1
2 b = 0
3 a = a ^ b
4 b = a ^ b
5 a = a ^ b
6
7 print(a, b)
8
```

0 0



0 1

1 0

13

```
1 def fun(x):
2     if x % 2 == 0:
3         return 1
4     else:
5         return 2
6
7
8 print(fun(fun(2)))
9
```

None

1

o código causará um erro de tempo de execução



2

## Pergunta 14

Dê uma olhada no snippet e escolha a afirmação **verdadeira**:

```
1 | nums = [1, 2, 3]
2 | vals = nums
3 | del vals[:]
4 |
```

o snippet causará um erro de tempo de execução



`nums` e `vals` têm o mesmo comprimento

`vals` é maior que `nums`

`nums` é maior que `vals`

## Pergunta 15

Qual é a saída do seguinte trecho de código se o usuário digitar duas linhas contendo `3` e `2` respectivamente?

```
1 | x = int(input())
2 | y = int(input())
3 | x = x % y
4 | x = x % y
5 | y = y % x
6 | print(y)
7 |
```

`1`

`2`

`3`



`0`



## Pergunta 16

Qual é a saída do seguinte trecho de código se o usuário digitar duas linhas contendo 3 e 6 respectivamente?

```
1 y = input()
2 x = input()
3 print(x + y)
4
```

36

6

3



63

## Pergunta 17

Qual é o resultado do seguinte trecho de código?

```
1 print("a", "b", "c", sep="sep")
2
```

asepbsepsep

a b c

abc



asepbsep

## Pergunta 18

Qual é o resultado do seguinte trecho de código?

```
1 | x = 1 // 5 + 1 / 5
2 | print(x)
3 |
```

0

0.5

0.4



0.2

## Pergunta 19

Supondo que `my_tuple` é uma tupla criada corretamente, o fato de que as tuplas são imutáveis significa que a seguinte instrução:

```
1 | my_tuple[1] = my_tuple[1] + my_tuple[0]
2 |
```

pode ser ilegal se a tupla contiver strings



é ilegal

está totalmente correto

pode ser executado se, e somente se, a tupla contiver pelo menos dois elementos

## Pergunta 20

Qual é a saída do seguinte trecho de código se o usuário digitar duas linhas contendo 2 e 4 respectivamente?

```
1 x = float(input())
2 y = float(input())
3 print(y ** (1 / x))
4
```

1.0

0.0



2.0

4.2

## Pergunta 21

Qual é a saída do seguinte snippet?

```
1 dct = {'one': 'two', 'three': 'one', 'two': 'three'}
2 v = dct['three']
3
4 for k in range(len(dct)):
5     v = dct[v]
6
7 print(v)
8
```

{'one', 'two', 'three'}

two



one

## Pergunta 22

Quantos elementos a lista `lst` contém?

```
1 | lst = [i for i in range(-1, -2)]  
2 |
```

`um`



`zero`

`dois`

`três`

## Pergunta 23

Qual das seguintes linhas invoca **corretamente** a função definida abaixo? (Selecione **duas** respostas)

```
1 | def fun(a, b, c=0):  
2 |     # Cuerpo de la función.  
3 |
```



`fun(0, 1, 2)`



`fun(b=0, a=0)`

`fun(b=1)`

`fun()`

Qual é a saída do seguinte snippet?

```
1 def fun(x, y):
2     if x == y:
3         return x
4     else:
5         return fun(x, y-1)
6
7
8 print(fun(0, 3))
9
```

1

o snippet causará um erro de tempo de execução

2



3

## Pergunta 25

Quantas estrelas (\*) o fragmento a seguir envia para o console?

```
1 i = 0
2 while i < i + 2 :
3     i += 1
4     print("#")
5 else:
6     print("#")
7
```

1

2



o snippet entrará em um loop infinito, imprimindo uma estrela por linha

## Pergunta 26

Qual é a saída do seguinte snippet?

```
1 | tup = (1, 2, 4, 8)
2 | tup = tup[-2:-1]
3 | tup = tup[-1]
4 | print(tup)
5 |
```



4

(4,)

44

(4)

## Pergunta 27

Qual é a saída do seguinte snippet?

```
1 | dd = {"1": "0", "0": "1"}
2 | for x in dd.vals():
3 |     print(x, end="")
4 |
```

1 0

0 0



o código está errado (o objeto `dict` não tem o método `vals()`)

0 1

## Pergunta 28

Qual é a saída do seguinte snippet?

```
1 | dct = {}  
2 | dct['1'] = (1, 2)  
3 | dct['2'] = (2, 1)  
4 |  
5 | for x in dct.keys():  
6 |     print(dct[x][1], end="")  
7 |
```



21

(2,1)

(1,2)

## Pergunta 29

Qual é a saída do seguinte snippet?

```
1 | def fun(inp=2, out=3):  
2 |     return inp * out  
3 |  
4 |  
5 | print(fun(out=2))  
6 |
```

6



4

O trecho é errôneo e causará SyntaxError

## Pergunta 30

Quantos hashes (#) o fragmento a seguir envia para o console?

```
1 lst = [[x for x in range(3)] for y in range(3)]
2
3 for r in range(3):
4     for c in range(3):
5         if lst[r][c] % 2 != 0:
6             print("#")
7
```



três

nove

seis

Qual é a saída do código a seguir se o usuário digitar um 0?

```
1 try:
2     value = input("Digite um valor: ")
3     print(int(value)/len(value))
4 except ValueError:
5     print("Entrada ruim...")
6 except ZeroDivisionError:
7     print("Entrada muito ruim...")
8 except TypeError:
9     print("Muito, muito ruim entrada...")
10 except:
11     print("Boooo!")
12
```

Boooo!

1.0



0.0



## Pergunta 32

Qual é o comportamento esperado do programa a seguir?

```
1 | try:
2 |     print(5/0)
3 |     break
4 | except:
5 |     print("Desculpe, algo deu errado...")
6 | except (ValueError, ZeroDivisionError):
7 |     print("Muito ruim...")
8 |
```



O programa causará uma exceção `SyntaxError`.

O programa causará uma exceção `ValueError` e gera uma mensagem de erro padrão.

O programa gerará uma exceção tratada pela primeira vez no bloco `except`.

## Pergunta 33

Qual é o comportamento esperado do programa a seguir?

```
1 | foo = (1, 2, 3)
2 | foo.index(0)
3 |
```



O programa causará uma exceção `ValueError`.

O programa causará uma exceção `TypeError`.

O programa causará uma exceção `SyntaxError`.

## Pergunta 34

Qual dos seguintes snippets mostra a maneira correta de lidar com várias exceções em uma única cláusula *except*?

```
1 # A:
2 except (TypeError, ValueError, ZeroDivisionError):
3     # Algo de código.
4
5 # B:
6 except TypeError, ValueError, ZeroDivisionError:
7     # Algo de código.
8
9 # C:
10 except: (TypeError, ValueError, ZeroDivisionError)
11     # Algo de código.
12
13 # D:
14 except: TypeError, ValueError, ZeroDivisionError
15     # Algo de código.
16
17 # E:
18 except (TypeError, ValueError, ZeroDivisionError)
19     # Algo de código.
```

```
10 except (TypeError, ValueError, ZeroDivisionError):
11     # Algo de código.
12
13 # D:
14 except: TypeError, ValueError, ZeroDivisionError
15     # Algo de código.
16
17 # E:
18 except (TypeError, ValueError, ZeroDivisionError)
19     # Algo de código.
20
21 # F:
22 except TypeError, ValueError, ZeroDivisionError
23     # Algo de código.
24
```



Somente A

A e B

B e C

D e E

## Pergunta 35

O que acontecerá quando você tentar executar o código a seguir?

```
1 | print(¡Hola Mundo!)  
2 |
```



O código gerará a exceção *TypeError*.

O código gerará a exceção *AttributeError*.



O código gerará a exceção *SyntaxError*.

O código irá imprimir `¡Hola Mundo!` no console.

O código gerará a exceção *ValueError*.